

Plataforma Web Inteligente para la Creación Automatizada de Landing Pages



Web**Landing**Suite

Integrantes del grupo

- Juan Carlos Agüero Carcamo
- Marcos Orellana
- Isaac Gonzale

Índice

Índice.....	2
Introducción.....	4
Contexto del proyecto.....	5
Antecedentes y motivación.....	5
Origen del problema.....	5
Publico objetivo general.....	5
Estado del arte y Tabla de Homologación.....	6
Descripción del problema u oportunidad.....	7
Definición del problema Central.....	7
Descripción del Impacto.....	7
Análisis de Causas y efectos: Árbol del Problema.....	7
Conclusión: Enfoque de la solución.....	9
Descripción del mercado objetivo.....	9
Objetivo general y objetivos específicos.....	10
Objetivo General.....	10
Objetivos Específicos.....	11
Coherencia y Alineación del Proyecto.....	11
Situación inicial del cliente (AS-IS).....	12
Contexto de la situación Actual.....	12
Descripción del Proceso Actual (Paso a Paso).....	12
Puntos de Dolor identificados en el Proceso.....	13
Diagrama del Proceso Actual (Flujo AS-IS).....	13
Planificación.....	15
Estructura del Trabajo.....	15
Inicio y Planificación.....	15
Servicios Cloud a utilizar (IaaS / PaaS / SaaS).....	16
Justificación de la Arquitectura Cloud.....	16
Frontend (Capa de Presentación - PaaS).....	17
Backend (Capa de Lógica y API - PaaS).....	17
Motor Inteligencia Artificial (SaaS).....	17
Conceptualización (solución propuesta – versión preliminar).....	17
Descripción de la solución (Modelo TO-BE).....	17
Funcionalidades principales:.....	18
Entregables de la Etapa de Construcción.....	19
Consideraciones de Calidad y Seguridad.....	19
Diagrama de Arquitectura de la Solución (TO-BE).....	20
Alcance, supuestos y restricciones.....	21
Alcance del Proyecto.....	21
Fuera de Alcance (Que no incluye el sistema).....	21
Entregables del Proyecto.....	22
Presupuestos.....	22
Restricciones.....	22

Metodología de desarrollo y gestión del proyecto.....	23
Documentos y diagramas de diseño de la solución.....	24
1. Historias de Usuario (Backlog Ágil).....	24
Módulo 1: Portal de Autenticación.....	24
Módulo 2: Gestión de Planes y Pagos.....	24
Módulo 3: Motor de IA y Creación.....	24
Módulo 4: Gestión de Proyectos (Dashboard).....	25
Módulo 5: Auditoría y Trazabilidad (Logs).....	25
Modulo 6: Gestion de Usuario.....	25
2. Diagrama de Casos de Uso y Actores.....	25
3. Modelo de Datos (Diagrama Entidad-Relación).....	26
Explicación del Modelo.....	27
Arquitectura de software y patrones de diseño.....	28
Tecnologías, lenguaje y dependencias (stack definitivo).....	29
Configuración de servidor de producción (paso a paso).....	30
Estado de avance del desarrollo (evidencia de producto final).....	31
Integraciones necesarias (si aplica).....	32
Conclusión.....	33
Anexos.....	34

Introducción

En la actual era digital, poseer una presencia en internet ha dejado de ser un lujo para convertirse en un requisito fundamental para la supervivencia y el crecimiento de cualquier emprendimiento, profesional independiente o microempresa. Una página web actúa como la principal carta de presentación y el canal de ventas más directo en el mercado global. Sin embargo, históricamente, la creación de un sitio web ha estado ligada a procesos complejos que demandan conocimientos especializados en diseño y programación, o en su defecto, una inversión económica significativa para contratar a agencias o terceros. Hoy en día, el vertiginoso avance de la Inteligencia Artificial está transformando radicalmente este panorama, abriendo la puerta a la automatización de tareas que antes requerían horas de trabajo manual, y democratizando el acceso a herramientas tecnológicas de alto nivel para usuarios sin un perfil técnico.

Es precisamente en esta brecha donde surge la motivación y oportunidad de este proyecto. Muchos microempresarios y trabajadores independientes se enfrentan a la frustración de necesitar una página de aterrizaje (*landing page*) rápida para validar una idea de negocio o captar clientes, pero se ven frenados por la pronunciada curva de aprendizaje de los constructores de sitios tradicionales (CMS) o por los prohibitivos costos del desarrollo a medida. Para dar respuesta a esta problemática, se construirá una plataforma web innovadora que automatiza el diseño web. El sistema permitirá a los clientes completar un formulario intuitivo con la información de su negocio y, mediante el uso estratégico de Inteligencia Artificial, generará automáticamente el código de una página web lista para usar. La gran innovación radica en ofrecer opciones escalonadas: el usuario podrá elegir entre distintos niveles de calidad y diseño (básico, intermedio o premium), pagando un precio justo y proporcional al nivel de sofisticación deseado.

El presente informe detalla exhaustivamente la planificación, diseño y viabilidad de esta solución tecnológica. En las siguientes páginas, el lector encontrará en primer lugar la definición formal del proyecto, incluyendo los objetivos generales y específicos que guiarán al equipo desarrollador. Posteriormente, se profundizará en el contexto que justifica la necesidad comercial de la plataforma, seguido de la descripción de la metodología de trabajo híbrida adoptada para asegurar entregas continuas y de calidad. Finalmente, el documento expondrá las especificaciones técnicas del sistema, abarcando la arquitectura de software elegida, los lenguajes de programación y la infraestructura que darán vida a este motor inteligente de generación web.

Contexto del proyecto

Antecedentes y motivación

En el ecosistema comercial actual, la digitalización ha dejado de ser una ventaja competitiva para transformarse en un requisito básico de supervivencia. Una presencia en internet solida válida la credibilidad de un negocio y actúa como su principal canal de captación de clientes. Sin embargo, el desarrollo web tradicional sigue siendo un proceso inherentemente técnico y costoso. Para un microempresario o un trabajador independiente, destinar gran parte de su capital inicial a contratar una agencia de desarrollo, o invertir semanas intentando aprender a programar, resulta inviable. La principal motivación de este proyecto es democratizar el acceso a la tecnología web, utilizando el auge de la inteligencia artificial para reducir dramáticamente las barreras técnicas y económicas. Se busca empoderar a los creadores de negocios entregándoles una herramienta capaz de generar una landing page profesional en minutos, permitiendo enfocar su tiempo y recursos en lo que realmente importa: hacer crecer su negocio.

Origen del problema

La problemática se detectó a través de la observación directa del mercado de trabajadores independientes y microempresas. Es común observar que muchos emprendedores emergentes dependen exclusivamente de perfiles en redes sociales (como Instagram o Facebook) para vender sus servicios, debido a la frustración que les genera intentar crear una página web. Al indagar en estas experiencias, se evidencia que los usuarios no técnicos se sienten abrumados ante la pronunciada curva de aprendizaje de los constructores de sitios actuales. Prometen ser fáciles de usar, pero en la práctica requieren nociones de diseños, estructura y configuración de dominios que el usuario promedio no posee, terminando en el abandono del proyecto o en páginas con un diseño deficiente que perjudican la imagen de la marca.

Publico objetivo general

El sistema está dirigido principalmente a microempresarios, profesionales independientes (freelancer), consultores y pequeñas empresas (pymes). Se trata de usuarios con nulos o bajos conocimientos en programación y diseño web, que cuentan con presupuesto limitado pero que tienen la necesidad urgente de lanzar una página de aterrizaje estética para validar una idea de negocio, promocionar un servicio o capturar datos de clientes potenciales (leads).

Estado del arte y Tabla de Homologación

Actualmente, la necesidad de tener una página web se resuelve de tres formas principales: contratando desarrollo a medida (inaccesible por precio), utilizando gestores de contenido complejos (como WordPress, que requieren mantenimiento técnico constante), o mediante constructores visuales (Site Builders). Si bien estas herramientas existen, no resuelven el problema de fondo para este nicho específico, ya que obligan al usuario a suscribirse a planes mensuales costosos o a diseñar desde cero sobre lienzos en blanco.

A continuación, se presenta la tabla de homologación comparando las soluciones actuales del mercado frente a la propuesta de este proyecto:

Solución Competidor /	Que hace	Ventajas	Desventajas	Que haremos distinto (Nuestra propuesta)
Constructores Visuales	Permite crear sitios web mediante interfaces de arrastrar y soltar.	Gran cantidad de planillas. Incluye hosting. No requiere saber programar.	Altas cuotas de suscripción mensual obligatorias. Curva de aprendizaje para lograr un buen diseño. El código no pertenece al usuario.	Generación por IA y pago único: El usuario no diseña nada manualmente; llena un formulario y la IA hace el trabajo. Se cobra un pago único por la generación de código descargable, eliminando los amarres de suscripción mensuales.
Gestores de Contenido (WordPress)	Sistema de gestión de contenido (CMS) altamente personalizable mediante temas y plugins.	Software base gratuito. Control total sobre el sitio y el servidor. Escalabilidad infinita.	Requiere conocimientos técnicos para configurar servidores, bases de datos y seguridad. Alto costo de mantenimiento y riesgo de vulnerabilidades si no se actualiza.	Simplicidad estética: Entregaremos archivos HTML/CSS puros y estáticos, listos para subir a cualquier hosting básico sin bases de datos complejas ni necesidad de mantenimiento o actualizaciones de seguridad.
Generadores de Código IA	Generan bloques de código HTML/CSS a partir de descripciones	Gratuitos o de muy bajo costos. Respuesta instantánea. Flexibilidad total en lo que se puede	Entregan código crudo, a menudo incompleto o con errores de sintaxis. El usuario debe saber cómo	Ingeniería de Prompts invisible y escalonadas: El usuario no redacta prompts; el sistema abstrae esa complejidad. El backend ensambla, limpia y valida el

	de texto libre (prompts).	pedir.	ensamblar los archivos, vincular el CSS y probarlo.	código generado en 3 niveles de calidad (básico, intermedio, premium) entregando un producto final listo para usar.
--	---------------------------	--------	---	---

Descripción del problema u oportunidad

Definición del problema Central

Actualmente, la creación de páginas web profesionales exige altos conocimientos técnicos o una inversión económica inicial muy elevada, lo que provoca microempresarios y profesionales independientes, credibilidad y oportunidades de venta en el mercado.

Descripción del Impacto

El impacto de esta problemática se manifiesta en múltiples dimensiones operativas y financieras para el usuario objetivo. En términos de tiempo, los emprendedores desperdician semanas intentando aprender a utilizar constructores de sitios o gestores de contenido (CMS), desviando su atención del núcleo de su negocio. En términos de costos, las alternativas se reducen a contratar agencias de desarrollo cuyas tarifas de cientos o miles de dólares resultan privativas para un negocio naciente o asumir costosas suscripciones mensuales en plataformas que retienen la propiedad del código (Vendor Lock-in).

Adicionalmente, el intento de “hacerlo uno mismo” sin conocimientos previos genera un alto margen de errores y una mala experiencia de usuario (UX), páginas que cargan lento, enlaces rotos y diseños que no se adaptan a dispositivos móviles (falta de Responsive Design). El efecto final de este impacto es la pérdida de conversiones (leads), un cliente potencial que visita un sitio web deficiente percibe falta de profesionalismo, lo que traduce en ventas perdidas y un estancamiento del crecimiento comercial.

Análisis de Causas y efectos: Árbol del Problema

Para desglosar estructuralmente esta situación, se ha elaborado el siguiente Árbol de Problemas, donde el tronco representa el problema central, las raíces corresponden a las causas que lo originan, y las ramas ilustran los efectos o consecuencias que el usuario sufre en la actualidad.

(Tronco) Problema Central: Imposibilidad de microempresarios y profesionales independientes para obtener una landing page funcional, profesional y a un costo accesible de manera ágil.



(Raíces) Causas (¿Por qué ocurre?)

1. **Barra técnica (Curva de aprendizaje):** Herramientas actuales (CMS y constructores visuales) exigen conocimientos subyacentes de diseño web, HTML/CSS, dominios y alojamiento de servidores.
2. **Barrera financiera (Costos prohibitivos):** El desarrollo de software a medida y el diseño UI/UX personalizado tienen un alto costo de mercado por horas de trabajo humano.
3. **Modelo de negocio restrictivo:** Las plataformas “gratuitas” o de bajo costo ocultan barreras de pago (suscripciones obligatorias mensuales) para poder publicar o conectar un dominio propio.
4. **Falta de abstracción en herramientas de IA:** Los actuales generadores de código por IA (como ChatGPT) entregan código crudo que el usuario no técnico no sabe cómo compilar, enlazar ni desplegar.

(Ramas) Efectos (¿Que consecuencias trae?):

1. **Dependencias tecnológicas riesgosas:** Los negocios operan exclusivamente a través de perfiles en redes sociales, quedando a merced de cambios de algoritmos y sin control real sobre su audiencia o sus datos.
2. **Fuga de capital operativo:** Pago de suscripciones mensuales de plataformas web que el usuario finalmente no utiliza porque no logró terminar de configurar el sitio.
3. **Deterioro de la imagen de marca:** Despliegue de sitios web “caseros” con errores de diseño visual, mala ortografía o estructuras confusas que generan desconfianza en el cliente final.
4. **Estancamiento comercial:** Imposibilidad de lanzar campañas de marketing digital (Google Ads, Facebook Ads) de manera efectiva al no contar con una página de aterrizaje (Landing Page) optimizada para la conversión.

Conclusión: Enfoque de la solución

La solución tecnológica propuesta en este proyecto (Plataforma Web Inteligente para la Creación Automatizada de Landing Page) atacará directamente las causas raíces 1 y 4 (Barrera técnica y falta de abstracción) y la causa 2 (Barrera financiera). Al delegar toda la carga de diseño estructural, maquetación y programación a un motor inteligencia artificial en el backend (Spring Boot), el usuario ya no necesita aprender a usar un constructor visual, sólo debe llenar un formulario. Asimismo, al post procesar este código y entregarlo listo en archivos descargables bajo un modelo de pago único y escalonado (básico, intermedio, premium), se elimina la barrera económica y la dependencia de suscripciones, resolviendo el problema de raíz y mitigando por completo sus efectos negativos en el mercado.

Descripción del mercado objetivo

El cliente objetivo (B2B/B2C) está compuesto por microempresarios, freelancers, consultores y pymes emergentes. Son personas que necesitan digitalizarse urgentemente pero no tienen conocimientos de programación ni presupuesto para un equipo técnico

Necesidades:

- **Inmediatez:** Lanzar una Landing Page funcional en minutos.
- **Abstracción Técnica:** Obtener código limpio (HTML/CSS) sin escribir ni una línea, mediante un formulario guiado por IA.
- **Flexibilidad:** Pagar lo justo según el nivel de diseño que requieran en el momento (básico, intermedio, premium), sin amarrarse a suscripciones mensuales.

Dolores:

- **Altos Costos:** Pagar a agencias o desarrolladores está fuera de su presupuesto inicial.
- **Pérdida de tiempo:** Los CMS "fáciles" (como Wix o WordPress) en realidad tienen una curva de aprendizaje alta, desconfiguran el diseño y desenfocan al usuario de su negocio principal.

Escenario de uso:

- **Cuándo y dónde:** El usuario ingresa desde su computador (en casa u oficina) en momentos de urgencia comercial (ej. un psicólogo que decide organizar un taller online para este fin de semana y necesita urgente una página para poner el link de inscripción en su Instagram, o un emprendedor que quiere probar si su nueva idea de negocio tiene demanda real antes de gastar dinero). Ingresa a la plataforma, completa sus datos comerciales, elige el nivel de diseño que se ajusta a su bolsillo, y en menos de 5 minutos descarga su web lista para usar.)
- **Usuario 1 (Principal - El emprendedor/Freelancer):**

Características: Profesional de 25 a 50 años. Experto en su rubro (ej. repostería, consultoría legal) pero con nulos conocimientos en programación o diseño UI/UX. Su presupuesto es limitado y su tiempo es escaso, ya que él mismo gestiona todas las áreas de su negocio. Es quien toma la decisión directa de usar y pagar por la herramienta.

Necesidad Principal: Necesito tener una página web hoy mismo para que mi negocio se vea profesional y pueda recibir contactos, pero solo tengo un presupuesto de \$X y no tengo tiempo para aprender a usar un software complejo. Necesito que alguien (o algo) lo haga por mí rápido.

- **Usuario 2 (Secundario - El Gestor de Marketing/Agencia Pequeña):**

Características: Profesional del área de marketing, *Community Manager* o trabajador de una agencia pequeña. Conoce conceptos de diseño y web, pero no es programador.

Necesidad Principal: Necesito generar múltiples versiones de *landing pages* rápidamente para probar diferentes campañas (A/B testing) para mis distintos clientes, sin tener que saturar al equipo de desarrollo con peticiones menores. Necesito un generador de código confiable y de buena calidad para agilizar mi trabajo diario.

Objetivo general y objetivos específicos

Objetivo General

Desarrollar una plataforma web SaaS (Software as a Service) impulsada por inteligencia artificial que automatice la maquetación y codificación de páginas de aterrizaje (Landing Page) escalonadas en calidad y precio, con el fin de permitir a microempresarios, profesionales independientes y pequeñas agencias establecer una presencia digital profesional de forma inmediata, accesible y sin requerir conocimientos previos de programación.

Objetivos Específicos

Para dar cumplimiento cabal al objetivo general y asegurar la construcción estructural del producto de software, se establece las siguientes metas concretas y medibles, divididas según las capas de la arquitectura del proyecto:

1. Modelar e implementar una base de datos relacional y una arquitectura de servidor multicapa (utilizando Spring Boot y java) que garantice el registro seguro de usuarios, la persistencia de los proyectos generados y la gestión del modelo de cobro simulado.
2. Diseñar e integrar un motor algorítmico de Prompt Engineering en el backend que se comunique con una API de inteligencia artificial externa, estructurando tres niveles de complejidad de peticiones (Básico, intermedio, premium) para abstraer al usuario de la redacción de instrucciones técnicas.
3. Construir una interfaz de usuario web interactiva y dinámica (empleando React y TailWind CSS) centrada en la usabilidad, que guíe al cliente a través de un formulario inteligente de captura de requerimientos comerciales en menos de 10 pasos.
4. Desarrollar e implementar un módulo de post procesamiento en el servidor que reciba el código crudo generado por la IA, lo limpie, valide su renderizado (HTML/CSS) y lo empaquete en archivos descargables listos para su publicación inmediata.

Coherencia y Alineación del Proyecto

La definición de estos objetivos garantiza el éxito del proyecto, ya que su cumplimiento secuencial resuelve directamente el problema central detectado. El objetivo 3 ataca la causa de la alta curva de aprendizaje (barrera técnica) al reemplazar los constructores visuales complejos por un formulario guiado. El objetivo 2 y 4 resuelven la falta de abstracción de las IAs actuales, ya que el sistema se encargará de estructurar, limpiar y validar el código, entregando un producto terminado. Finalmente, la integración de toda la arquitectura (objetivo 1) sostiene el modelo de negocio de pago escalonado, eliminando la barrera de los costos prohibitivos y democratizando el acceso a la tecnología web para el mercado objetivo.

Situación inicial del cliente (AS-IS)

Contexto de la situación Actual

En el escenario actual, cuando un micro empresario o profesional independiente (freelance) detecta la urgencia de contar con una página web para validar un negocio o captar clientes, se enfrenta a un ecosistema tecnológico que no está diseñado para su nivel de conocimientos ni para su presupuesto. Dado que la contratación de una agencia de desarrollo a medida queda descartada de inmediato por la barrera económica, el usuario se ve forzado a tomar la ruta del “Hágalo usted mismo” utilizando constructores de sitios visuales (CMS) o, más recientemente, intentando usar inteligencia artificial de texto crudo sin interfaz gráfica

Descripción del Proceso Actual (Paso a Paso)

1. **El usuario detecta la necesidad e inicia la búsqueda:** El emprendedor requiere una página de aterrizaje urgente. Busca en internet “como crear una página web gratis” y es bombardeado por publicidad de gestores de contenido complejos o constructores visuales.
2. **Luego, el usuario se registra y se enfrenta a la sección (Sobrecarga cognitiva):** Ingresa a una plataforma tradicional, crea una cuenta y se topa con un catálogo de cientos de plantillas genéricas. Pierda horas valiosas intentando decidir cuál se adapta a su rubro, sufriendo “parálisis por análisis”.
3. **El usuario intenta personalizar el diseño (El cuello de botella técnico):** Una vez elegida la plantilla, comienza el proceso manual de diseño utilizando herramientas de “arrastrar y soltar”. El problema ocurre cuando el usuario intenta cambiar márgenes, reemplazar imágenes o ajustar columnas, el diseño responsivo (la vista para celulares) se desconfigura por completo, exigiendo conocimientos de estructuración visual que no posee.
4. **El usuario redacta el contenido manualmente:** Sin ser experto en marketing o redacción comercial, el usuario debe inventar los textos persuasivos de su página, enfrentándose al síndrome del lienzo blanco, lo que retrasa aún más el lanzamiento.
5. **El problema crítico ocurre al intentar publicar (La barrera de Pago):** Tras días de frustración y un diseño ensamblado a medias, el usuario intenta publicar la web. En este punto, la plataforma le informa que la versión “gratuita” insertará anuncios invasivos de otras marcas y utilizará un subdominio genérico poco profesional. Para obtener los archivos reales o conectar su propio dominio, el sistema lo obliga a pagar una costosa suscripción mensual perpetua.

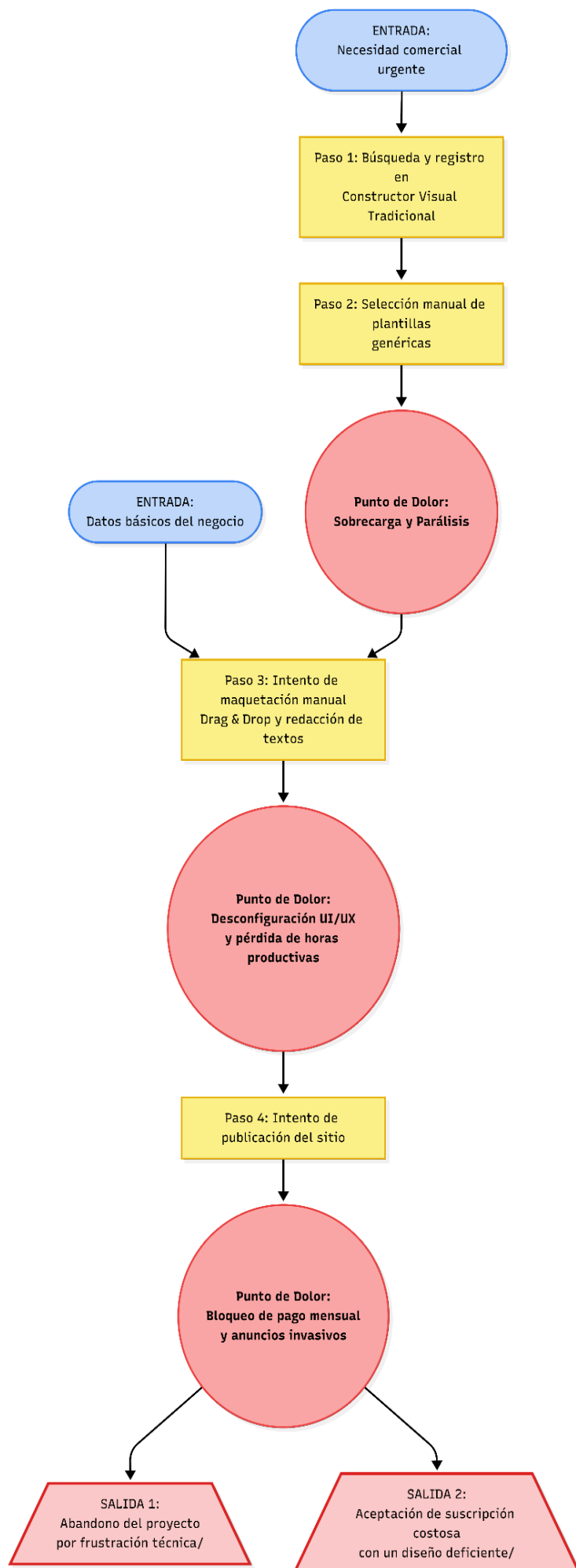
Puntos de Dolor identificados en el Proceso

- Desperdicio de tiempo productivo: El proceso consume entre 3 a 7 días hábiles de un emprendedor que debería estar vendiendo, no intentando maquetar interfaces.

- Falta de Abstracción: El sistema actual obliga al usuario a tomar decisiones técnicas (tipográficas, espaciados, estructura HTML invisible) para las cuales no está capacitado.
- Modelo de retención financiera: El cliente se da cuenta al final del proceso de que nunca será dueño del código de su página, quedando atado a mensualidades abusivas para que su sitio no sea dado de baja.

Diagrama del Proceso Actual (Flujo AS-IS)

A continuación, se esquematiza el flujo operativo descrito, evidenciando como las entradas del usuario terminan en salidas deficientes debido a los obstáculos del proceso:



Planificación

Estructura del Trabajo

Para garantizar la viabilidad del proyecto dentro de los plazos establecidos. La ejecución se ha estructurado bajo la metodología híbrida previamente justificada. El equipo de tres integrantes se dividirá las responsabilidades técnicas específicas: Integrante Juan Carlos Agüero (Especialista Frontend/UI), Integrante Marcos Orellana (Especialista Backend/BD) e Integrante Isaac Gonzales (Ingeniero de Prompts/IA)

A continuación, se detalla la lista de las 10 actividades principales que llevará el proyecto desde su concepción hasta su entrega final, desglosadas por fases, duración y responsables.

Inicio y Planificación

Esta fase es predictiva y busca cimentar las bases arquitectónicas del software.

1. Levantamiento y requerimientos y definición de arquitectura. Se definen las reglas de negocio, los endpoints de la API y las restricciones técnicas del sistema.
2. Modelamiento de Base de Datos y Diseño UI/UX. Creación del diagrama entidad-Relación y diseño visual de las pantallas en herramientas como figma.
3. Configuración de entornos y despliegue de la API base. inicialización del proyecto en Spring Boot, conexión a la base de datos y creación del esqueleto multicapa (controller, Service, repositorio).
4. Desarrollo de la interfaz de usuario y formulario dinámico. Construcción del Frontend en React/Vite, aplicando Tailwind CSS para el diseño responsivo del formulario de captura de datos.
5. Diseño, calibración y pruebas de Prompts Escalonados. Redacción y validación estricta de las 3 plantillas de IA (Básico, Intermedio, Premium) para asegurar que devuelvan HTML/CSS consistente.
6. Integración de API externa y lógica de post-procesamiento. Conexión segura desde el servidor a la IA externa y programación del algoritmo que limpia y formatea el código crudo generado.
7. Conexión Frontend-Backend y simulación de pagos. Consumo de los endpoints desde React, flujo de selección de planes y simulación de la pasarela de cobro.
8. Pruebas Integrales (QA) y validación de código generado. Depuración exhaustiva de errores (bugs), pruebas de estrés en la generación de páginas y validación de seguridad.
9. Despliegue en la nube (Puesta en producción). Migración del sistema desde los entornos locales (localhost) hacia plataformas en la nube (PaaS/IaaS) para su acceso público.

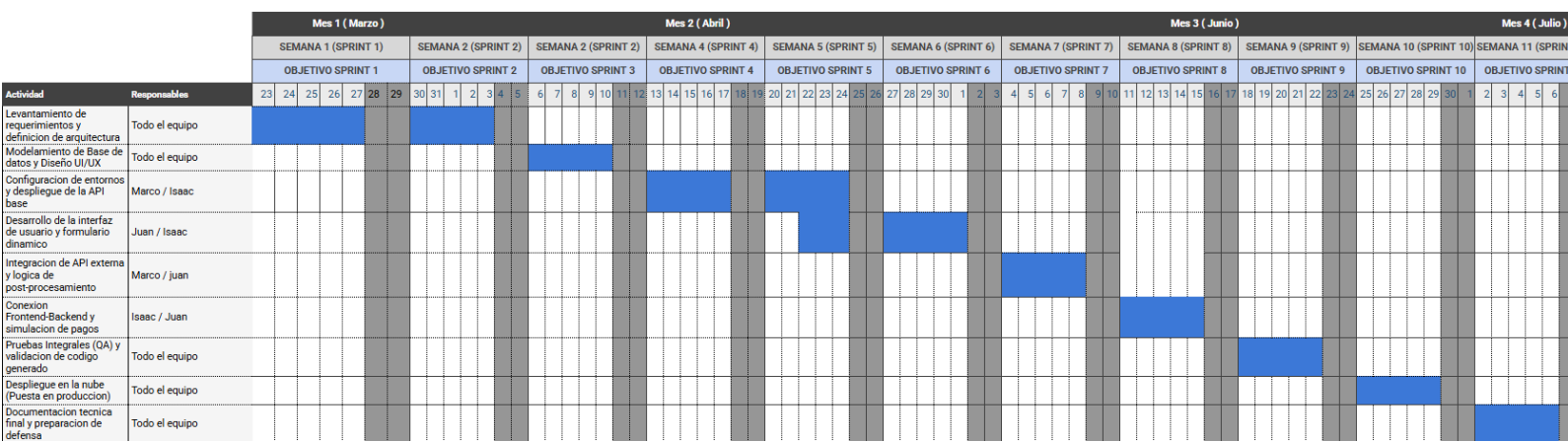
10.Documentación técnica final y preparación de defensa. Cierre del informe académico, manuales de usuario y ensayos para la presentación ante la comisión evaluadora.

Link

Carta

Gantt:

<https://docs.google.com/spreadsheets/d/1iY7XXo1Mn2V9pTDYK2O2z5PMAXIOYwQc7IA-M7dUQiyI/edit?usp=sharing>



Servicios Cloud a utilizar (IaaS / PaaS / SaaS)

Justificación de la Arquitectura Cloud

El despliegue y puesta en producción de la plataforma web se realizará íntegramente utilizando servicios en la nube (cloud computing). Se ha descartado el uso de servidores físicos locales y soluciones IaaS (Infraestructura como servicio, como Amazon EC2 puro) debido a la sobrecarga administrativa que conllevan. En su lugar, la arquitectura se basará estratégicamente en modelos PaaS (Plataforma como Servicio) y SaaS (Software como servicio). Esta decisión técnica garantiza una alta disponibilidad, permite la escalabilidad automática ante picos de usuarios generando sitios web, facilita el despliegue continuo directamente desde los repositorios de código, y optimiza los costos operativos, aprovechando las capas gratuitas y el pago por uso durante la fase de validación del proyecto.

Fronted (Capa de Presentación - PaaS)

- Servicio a utilizar: Vercel o Netlify (Plataforma como Servicio).
- Justificación: La interfaz de usuario, desarrollada como una Single Page Application (SPA) con React y Vite, requiere un entorno optimizado para la entrega de archivos estáticos. Vercel actúa como un PaaS que compila automáticamente el código fuente desde GitHub y lo distribuye a través de una Red de entrega de Contenidos global. Esto asegura tiempos de carga milisegundos para el cliente en cualquier parte del mundo, ofreciendo además certificados SSL automáticos para garantizar conexiones seguras (HTTPS).

Backend (Capa de Lógica y API - PaaS)

- Servicio a utilizar: Render o Railway (Plataforma como Servicio).
- Justificación: Para mantener la integridad de las cuentas de usuario, el registro de los planes seleccionados y el historial de transacciones, se requiere un motor relacional estricto (PostgreSQL). Utilizar un servicio administrado en la nube elimina la necesidad de gestionar copias de seguridad manuales, parches de seguridad o configuraciones de red. Garantiza que la persistencia de los datos sea robusta, tolerante a fallos y accesible de forma segura y exclusiva por el Backend.

Motor Inteligencia Artificial (SaaS)

- Servicio a utilizar: API de OpenAI, Anthropic o equivalente (Software como Servicio).
- Justificación: La plataforma no entrenará un modelo de lenguaje propio debido a las limitaciones de infraestructura (IaaS). En su lugar, consumirá un modelo de IA de frontera bajo el formato SaaS mediante peticiones HTTP. El servidor enviará los prompts escalonados (Básico, Intermedio, Premium) y la API externa retornará el código HTML/CSS procesado. Este modelo permite integrar inteligencia artificial de nivel mundial pagando fracciones de centavo por cada generación, manteniendo el costo operativo estrictamente ligado al uso real del sistema.

Conceptualización (solución propuesta – versión preliminar)

Descripción de la solución (Modelo TO-BE)

En contraposición al modelo actual (AS-IS), donde el usuario sufre una sobrecarga cognitiva intentando diseñar interfaces manualmente y lidiar con configuraciones de servidores, el modelo propuesto (TO-BE) abstrae y automatiza integralmente la

complejidad técnica. La plataforma funcionará como un intermediario inteligente (SaaS) que transforma requerimientos en lenguaje natural y datos estructurados en código fuente renderizable.

El nuevo flujo será el siguiente: el usuario entra a la plataforma web y, en vez de encontrarse con una página en blanco o muchas plantillas, verá un Formulario Inteligente que lo guiará paso a paso para conocer su negocio, como el nombre, rubro, colores, estilo de comunicación y servicios principales; luego de completarlo, podrá elegir el tipo de página que quiere (Plan Básico, Intermedio o Premium), y en ese momento el sistema se encargará de tomar la información, enviarla a una inteligencia artificial y generar la página automáticamente; en pocos minutos, el usuario verá una vista previa de su sitio web y, finalmente, después de confirmar el pago, podrá descargar el código de su página (HTML, CSS y JavaScript), listo para subirlo a cualquier hosting, quedando como dueño total de su sitio sin pagos mensuales.

Funcionalidades principales:

1. **Formulario Dinámico de Captura:** Interfaz fluida desarrollada en React que captura los requerimientos del cliente mediante validaciones en tiempo real, asegurando que la IA reciba el contexto exacto para generar un diseño coherente.
2. **Motor de Prompt Engineering Escalonado:** Núcleo lógico del servidor que clasifica la solicitud del usuario y selecciona automáticamente una de las tres plantillas de instrucciones restrictivas (Básico para HTML simple; Intermedio para diseño responsivo con Tailwind; Premium para animaciones y SEO básico), inyectando los datos del cliente antes de enviarlos a la IA externa.
3. **Módulo de Post-procesamiento y Sanitización:** Algoritmo encargado de recibir la respuesta cruda de la IA, limpiar cualquier texto conversacional ("Aquí tienes tu código..."), extraer estrictamente las etiquetas de maquetación y validar que la estructura del DOM no contenga errores críticos antes de mostrarla.
4. **Visor de Previsualización en Tiempo Real (Live Preview):** Un componente en la interfaz de usuario que renderiza de manera segura el código generado dentro de un entorno aislado (Iframe), permitiendo al cliente interactuar con el diseño antes de la compra.
5. **Simulador de Pasarela de Pago y Descarga:** Funcionalidad que procesa la selección del plan (facturación simulada) y habilita la creación dinámica de un archivo comprimido (.zip) con los activos digitales finales listos para descarga.
6. **Panel de Gestión de Proyectos (Historial):** Un *dashboard* privado donde los usuarios (especialmente útil para agencias o perfiles B2B) pueden visualizar el historial de las Landing Pages que han generado, acceder a descargas previas y revisar los recibos de sus transacciones.

Entregables de la Etapa de Construcción

Al finalizar el ciclo de desarrollo del proyecto, el equipo entregará los siguientes artefactos tangibles:

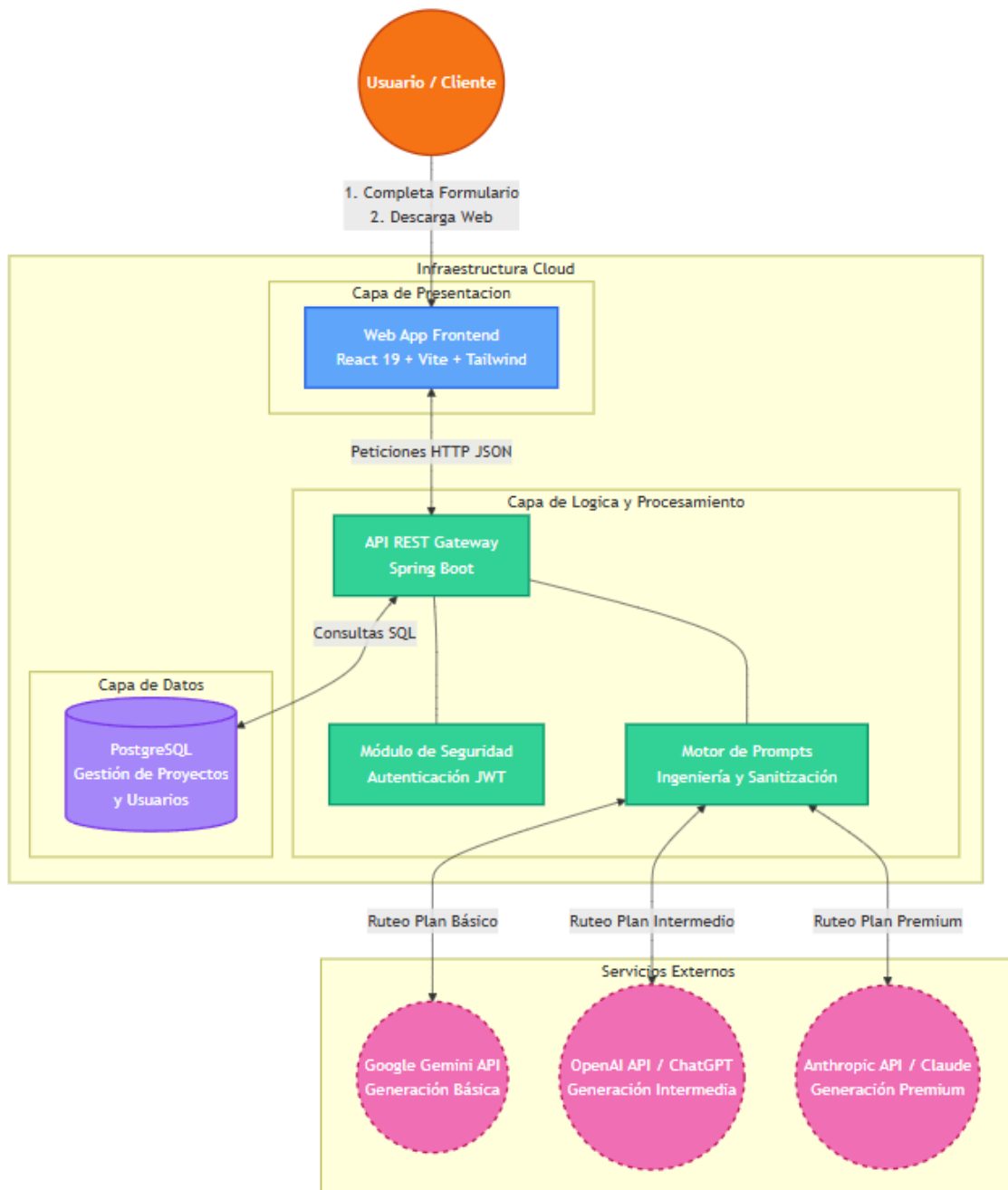
- Código Fuente del Frontend: Repositorio estructurado de la Single Page Application (React/Vite)
- Código Fuente del Backend: Repositorio de la API RESTful (Java con Spring Boot), incluyendo la lógica de integración con la IA.
- Script y modelo de la Base de Datos: Archivos DDL y DML para la generación de la estructura relacional (PostgreSQL).
- Documentación técnica: Manuales de despliegue, diccionarios de datos y documentación de los endpoints (Swagger/OpenAPI).

Consideraciones de Calidad y Seguridad

Al tratar con la generación automatizada de código mediante inteligencia artificial, la seguridad es un pilar no negociable.

- Autenticación y Autorización: El acceso a la plataforma y al panel de proyectos estará protegido mediante JSON Web Tokens (JWT), garantizando que las sesiones sean sin estado (stateless) y escalables en la nube.
- Validación y Prevención de XSS (Cross-Site Scripting): Dado que la IA genera código que luego será renderizado en el navegador del usuario, el módulo de post-procesamiento implementará librerías de sanitización estrictas en el backend para eliminar cualquier intento de inyección de scripts maliciosos o etiquetas no permitidas.
- Respaldo y Transaccionalidad: Las propiedades ACID de la base de datos relacional asegurarán que el registro de los pagos y la asociación de los proyectos a los usuarios no sufran inconsistencias. Adicionalmente, se configurarán respaldos automatizados (backups) diarios en la plataforma de alojamiento en la nube (DBaaS) para asegurar la persistencia ante desastres.

Diagrama de Arquitectura de la Solución (TO-BE)



Alcance, supuestos y restricciones

Alcance del Proyecto

El presente proyecto contempla el desarrollo integral (Frontend, Backend y Base de Datos) de una plataforma web SaaS (Software as a Service) para la generación automatizada de páginas de aterrizaje estáticas mediante inteligencia artificial. El alcance funcional se delimita a las siguientes capacidades operativas:

- Registro, autenticación y gestión de perfiles de usuario.
- Implementación de un formulario dinámico inteligente para la captura de requerimientos comerciales y preferencias visuales.
- Integración mediante API RESTful con un modelo de inteligencia artificial externo (ej. OpenAI o Anthropic) gestionada a través de un prompt de "Prompt Engineering" estructurado en tres niveles (Básico, intermedio y premium).
- Procesamiento, limpieza y validación en el servidor del código HTML, CSS Y JS generado por la IA.
- Renderizado de una pre visualización interactiva (Live Preview) en un entorno web aislado y seguro.
- Simulación del flujo de facturación según el plan seleccionado.
- Generación y entrega de los archivos finales empaquetados (formato .zip) listos para su descarga inmediata.

Fuera de Alcance (Que no incluye el sistema)

Para mantener la viabilidad del proyecto dentro de los plazos y recursos disponibles, se excluyen explícitamente las siguientes funcionalidades:

- Alojamiento web (Hosting) de los sitios generados: La plataforma entregara los archivos fuentes (HTML/CSS/JS), pero no proveerá el servicio de alojamiento público ni la compra/configuración de dominios personalizados (.com, .cl) para el cliente final.
- Gestor de Contenidos: No se desarrollará un panel de administración para que el usuario edite la página web después de haberla descargado. Se trata de páginas estáticas de un solo uso o generación.
- Pasarelas de pagos reales: La plataforma no integrará transacciones bancarias reales (Webpay, Stripe, paypal en producción). El flujo de cobro por los planes será netamente con fines de demostración académica.
- Desarrollo de sistemas web complejos: La IA generará exclusivamente Landing Pages estáticas informativas. No se contempla la generación de tiendas online con carritos de compra, bases de datos propias para el cliente o sistema de inicio de sesión dentro de la página generada.

Entregables del Proyecto

Al término del ciclo de desarrollo, el equipo proveerá los siguientes componentes:

1. Producto de Software (MVP): Plataforma web funcional desplegada en un entorno de producción en la nube (PaaS/SaaS).
2. Código Fuente: Repositorios de control de versiones (Git), estructurados para Frontend y Backend.
3. Documentación Técnica: Informe final del proyecto de título, diagramas de arquitectura multicapa, modelo relacional de la base de datos (Entidad-Relación) y documentación técnica de los endpoints de la API (Swagger).

Presupuestos

Para la planificación y ejecución de este proyecto, se asume como cierto que:

- Disponibilidad Tecnológica: Las APIs de inteligencia artificial de terceros mantendrá su estabilidad, disponibilidad y mantendrá vigentes sus niveles gratuitos o de bajo costo para pruebas de desarrollo.
- Adopción del Usuario: Se asume que el mercado objetivo (microempresas y freelancers) posee un nivel de alfabetización digital básico, suficiente para interactuar con un formulario web, descargar un archivo comprimido (.zip) y seguir instrucciones simples para subirlo a un proveedor de hosting económico.

Restricciones

- Restricciones de Recursos Humanos: El equipo de desarrollador consta exclusivamente de 3 estudiantes de último año, quienes deben compatibilizar la carga horaria de la construcción de este software con sus respectivas prácticas profesionales en curso y otras responsabilidades académicas.
- Restricción Presupuestaria: El proyecto cuenta con un presupuesto equivalente a cero. Por ende, la infraestructura técnica dependerá del uso eficiente de las capas gratuitas (free tiers) de los servicios en la nube (vercel, render, postgresQL) y del manejo cuidadoso de los créditos limitados de la API de IA.
- Restricción Tecnológica (Alucinaciones de la IA): Al depender de Modelos Fundacionales Externos (LLMs), existe la restricción de imprevisibilidad en las respuestas. Esto obliga al equipo a invertir un tiempo considerable en imponer validaciones estrictas en la capa de negocio (Backend) para mitigar errores de sintaxis en el código generado.

Metodología de desarrollo y gestión del proyecto

Documentos y diagramas de diseño de la solución

Para dar forma al desarrollo de WebLandingSuite y asegurar que los requerimientos técnicos satisfagan las necesidades del modelo comercial, el diseño de la solución se ha estructurado en base a requerimientos ágiles (Historias de Usuarios) y su posterior abstracción técnica (Casos de uso). Esta documentación define la interacción entre los distintos perfiles de usuarios y las capas de nuestra arquitectura (Frontend en React y APIs en Spring Boot).

1. Historias de Usuario (Backlog Ágil)

Módulo 1: Portal de Autenticación

- HU01: Como visitante, quiero navegar por la Landing del producto para entender qué ofrece.
- HU02: Como visitante, quiero registrarme con mi correo y contraseña para crear mi cuenta.
- HU03: Como usuario, quiero iniciar sesión en el portal y recibir un token seguro (JWT) para acceder a mi cuenta.
- HU04: Como usuario, quiero actualizar mi perfil (nombre/apellido) desde la configuración de mi cuenta.

Módulo 2: Gestión de Planes y Pagos

- HU05: Como administrador, quiero gestionar (crear/editar/eliminar) los planes de diseño para actualizar la oferta.
- HU06: Como usuario, quiero ver los planes disponibles en el Frontend para elegir el que necesito.
- HU07: Como usuario, quiero registrar mi transacción de pago para activar la generación de mi proyecto.
- HU08: Como usuario, quiero consultar mi historial de pagos para llevar un control de mis gastos.

Módulo 3: Motor de IA y Creación

- HU09: Como usuario, quiero usar un formulario interactivo para ingresar el nombre, idea y colores de mi proyecto.
- HU10: Como sistema (Backend), quiero enviar los datos del proyecto FastAPI para que la IA procese y genere la web.

- HU11: Como usuario, quiero visualizar en el Frontend el resultado final de mi Landing Page generada.

Módulo 4: Gestión de Proyectos (Dashboard)

- HU12: Como usuario, quiero acceder a mi Dashboard para ver una lista con todos mis proyectos creados.
- HU13: Como usuario, quiero eliminar un proyecto antiguo para mantener mi espacio de trabajo organizado.
- HU14: Como administrador, quiero tener la capacidad de cambiar manualmente el estado de un proyecto (ej de “Procesando” a “Fallado”) por si ocurre un error.

Módulo 5: Auditoría y Trazabilidad (Logs)

- HU15: Como sistema, quiero registrar automáticamente eventos críticos (login, creación de proyecto) junto con la IP del usuario.
- HU16: Como administrador, quiero consultar el historial completo de eventos (Logs) para detectar actividades sospechosas o errores.

Modulo 6: Gestion de Usuario

- HU17: Como administrador, quiero visualizar el listado completo de usuarios registrados en la plataforma.

2. Diagrama de Casos de Uso y Actores

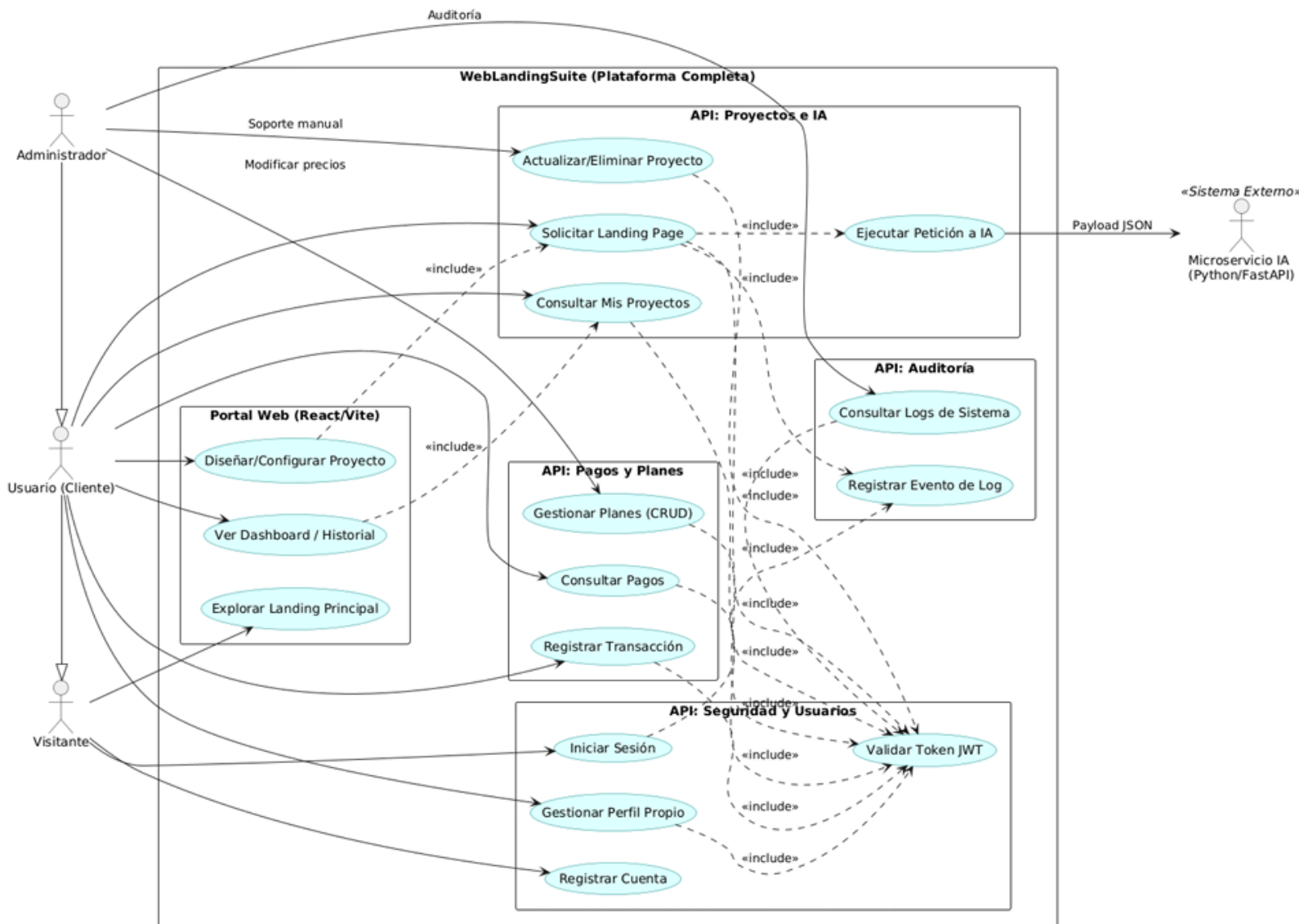
El siguiente diagrama modela las interacciones descritas en las HU frente a la arquitectura del sistema, delineando claramente las fronteras de las distintas APIs (Seguridad, Pagos, Proyectos y Auditoria)

Se definen cuatro actores principales:

1. Visitante: Interactúa únicamente en el Frontend para explorar el producto y registrarse.
2. Usuario (Cliente): Actor autenticado que interactúa de manera transversal con el sistema para configurar proyectos, gestionar pagos y visualizar su dashboard.
3. Administrador: Posee privilegios elevados para soporte manual, configuración de precios y consulta de logs de sistema.
4. Sistema Externo (Microservicio IA): Actor sistema (desarrollado en Python/FastAPI) encargado de recibir el Payload JSON y ejecutar la generación algorítmica de la página web.

Para garantizar la integridad y seguridad de la plataforma, el diseño arquitectónico establece que acciones críticas (como solicitar una Landing Page o consultar pagos)

dependen estrictamente de la validación del Token JWT mediante la relación <<include>>. Del mismo modo, estas transacciones gatillan la relación <<include>> hacia la API de Auditoría para registrar automáticamente el evento en los Logs del sistema.



3. Modelo de Datos (Diagrama Entidad-Relación)

- Propósito: Garantizar la persistencia estructurada, segura y escalable de la información de los clientes, sus transacciones financieras y los activos digitales generados por el motor de Inteligencia Artificial.

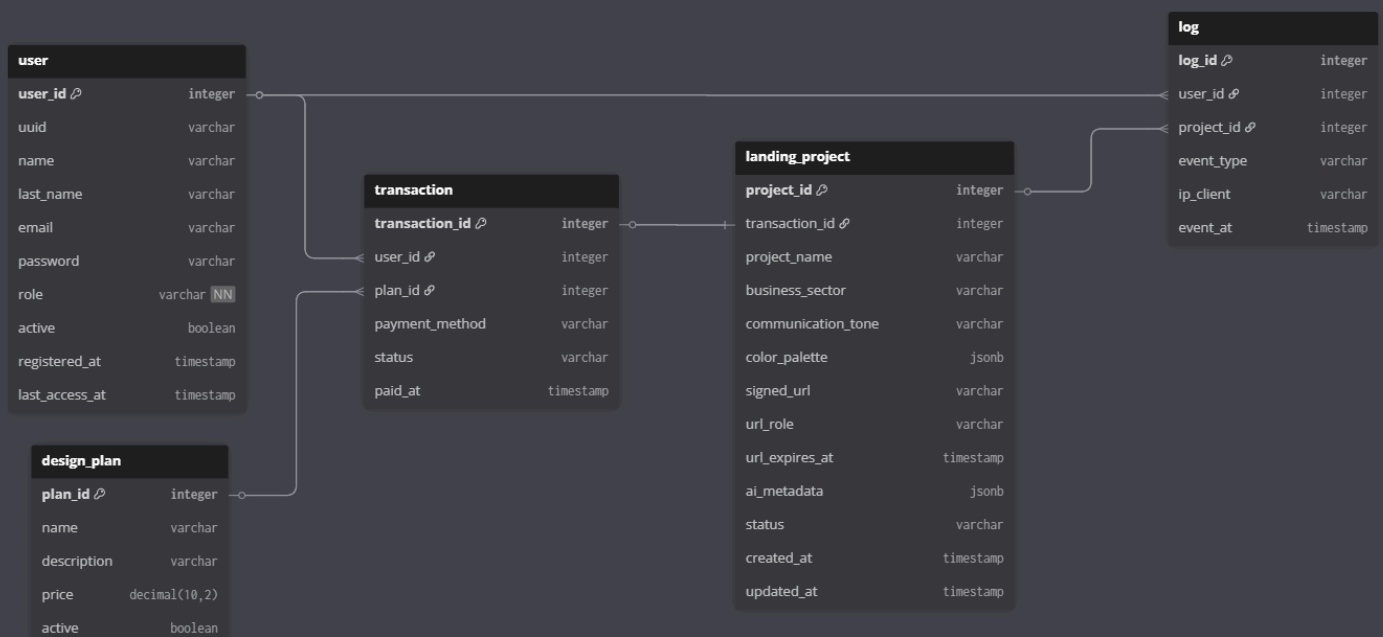
- Que representa: El esquema relacional de la base de datos que será alojada en PostgreSQL (mediante Neon), detallado las cinco entidades principales del sistema (user, design_plan, transaction, landing_project y log), sus atributos, tipos de datos y cardinalidad.
- Relación con el problema y objetivos: Este modelo da cumplimiento directo al objetivo específico 1 del proyecto. Al centralizar la información, se resuelve el problema de la pérdida de progreso que sufren los usuarios en plataformas tradicionales. Además, soporta el modelo de negocio escalonado al vincular estrictamente los proyectos generados con transacciones válidas y planes específicos.

Explicación del Modelo

El diseño de la base de datos se basa en el núcleo transaccional. La tabla central es transaction, la cual actúa como puente entre el cliente (user) y el servicio adquirido (design_plan). Una vez que una transacción es validada, el sistema habilita la persistencia en la tabla landing_project, la cual almacena todos los requerimientos de diseño enviados por el usuario.

Destacan dos decisiones arquitectónicas clave para este modelo:

1. **Flexibilidad para la IA:** En la tabla landing_project, los atributos color_palette y ai_metadata utilizan el tipo de datos jsonb. Esto permite almacenar estructuras de datos dinámicas y complejas devueltas por el microservicio en Python (FastAPI), sin necesidad de alterar el esquema relacional si los parámetros de la IA cambian en el futuro.
2. **Trazabilidad y Seguridad (Auditoría):** La tabla log se relaciona tanto con el usuario como con el proyecto, registrando eventos críticos (event_at). Esto protege la plataforma contra abusos y facilita el soporte técnico, asegurando la trazabilidad de cada petición al motor de generación web.



Arquitectura de software y patrones de diseño

1. Descripción de la Arquitectura General del Sistema

El proyecto “WebLandingSuite” implementa una arquitectura Cliente-Servidor de tipo SPA (Single Page Application) separada físicamente, donde el cliente (Frontend) y el servidor (Backend) se comunican a través de una API RESTful. Además, el sistema adopta un enfoque orientado a Microservicios en la capa de procesamiento avanzado, delegando tareas específicas de alta carga computacional (como la generación de contenido y estructuras web) a un servidor independiente de inteligencia artificial.

La arquitectura se divide en tres bloques principales

Tecnologías, lenguaje y dependencias (stack definitivo)

Configuración de servidor de producción (paso a paso)

**Estado de avance del desarrollo
(evidencia de producto final)**

Integraciones necesarias (si aplica)

Conclusión

Anexos